

```
# Install required packages (if not already installed)

# !pip install pandas scikit-learn matplotlib seaborn


# Step 1: Import necessary libraries

import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score


# Step 2: Load the dataset

df = pd.read_csv('/content/churn.csv') # Adjust path if needed


# Step 3: Basic data exploration

print("First 5 rows:")

print(df.head())

print("\nData types:")

print(df.dtypes)

print("\nMissing values:")

print(df.isnull().sum())


# Step 4: Data preprocessing

# Encode categorical columns

label_encoders = {}

for column in df.select_dtypes(include=['object']).columns:

    le = LabelEncoder()

    df[column] = le.fit_transform(df[column])

    label_encoders[column] = le
```

```
# Optional: Check correlation
```

```
plt.figure(figsize=(12, 8))
```

```
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap='coolwarm')
```

```
plt.title("Feature Correlation Heatmap")
```

```
plt.show()
```

```
# Step 5: Split the dataset
```

```
X = df.drop('Churn', axis=1) # Replace with correct target column name if different
```

```
y = df['Churn']
```

```
# Normalize features
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
# Step 6: Train a model
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
# Step 7: Evaluate the model
```

```
y_pred = model.predict(X_test)
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("\nAccuracy Score:")
```

```
print(accuracy_score(y_test, y_pred))
```

```
# Step 8: Feature Importance

importances = model.feature_importances_

feature_names = X.columns

indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 6))

sns.barplot(x=importances[indices], y=feature_names[indices])

plt.title("Feature Importances")

plt.show()
```